IOTA JU Redesign
Ritendu: CMS, Permissions and Content Data

Implementation handout

21 October 2026

1 Your job

You own the content models, administrator/editor permissions, validation, relationships, publication hooks and editor instructions. Payload is the content system and admin panel inside the Next.js application. PostgreSQL stores records. Your work lets the public pages read approved content without exposing drafts or private information.

In plain terms: a collection is a type of record, such as Project or Event. A global is one shared record, such as the homepage settings. A migration is a committed database change that can be applied safely to staging and production. A relationship is a link between records, such as a person and several team memberships.

2 What to install and configure

Install Git, the repository's supported Node.js LTS version (at least 20.18.1 for the current Payload requirement) and pnpm. Sourasish owns all root dependencies. Your first setup is:

```
pnpm install --frozen-lockfile
pnpm dev
```

Use only the database URL supplied for local development or staging. Do not add a new ORM, SQL client, API server or package unless the team approves it. Do not place database credentials in browser code or commit secrets. Ask Sourasish to change root packages or scripts.

Migration commands: Sourasish adds the Payload CLI script. After a schema change is ready, use `pnpmpayloadmigrate:create` to generate a migration and review/commit it. To apply the committed migrations in the selected environment, use `pnpmdb:migrate`. Never point these commands at production without the release procedure and orchestrator approval.

3 Folders and ownership

Files/folders you own	Purpose

`src/payload.config.ts`	Payload configuration after Sourasish completes P01. You own its collection/global registration and migration ordering.
`src/collections/Users.ts`	Private CMS accounts and Editor/Admin roles. Not public club-member profiles.
`src/collections/Projects.ts`, `src/collections/Events.ts`, `src/collections/People.ts`, `src/collections/Memberships.ts`	Main editorial records and the link between people, groups and years.
`src/collections/Milestones.ts`, `src/collections/Redirects.ts`	Historical timeline records and exact legacy path decisions.
`src/globals/Home.ts`, `src/globals/About.ts`, `src/globals/SiteSettings.ts`	Shared homepage/About/footer/navigation settings.
`src/access/`	Reusable role/access functions and tests.
`src/hooks/`	Publication event and relationship validation hooks.
`src/migrations/`, `src/payload-types.ts`	Committed schema changes and generated CMS types. Never hand-edit generated types.
`docs/content-contract.md`, `docs/editor-guide.md`	Agreed data shape and editor handover.
`tests/permissions.test.ts`, `tests/projects.test.ts`, `tests/events.test.ts`, `tests/memberships.test.ts`, `tests/relationships.test.ts`	Tests for your own access rules and data.

Other owners: Sourasish owns package/config/CI/public readers and starts Payload config before handing it to you. Shreyam owns the Media collection, storage adapter and import scripts; coordinate registration and schema migrations with him. Pritish owns page layout and design.

4 Shared data contract (B01)

Before building collections, create `docs/content-contract.md`. Use the real source inventory from Shreyam. Agree the shape with all three other members, especially public fields and media references. Names and slugs are not safe identity keys.

- **Project:** stable legacy key, title, slug, original summary/body, explicit sort order, lifecycle, separate award/program label and award year, media, reports and verified people links. Preserve the known 11-card order, AGV feature and linked historical detail pages.
- **Event:** stable key, title, slug, original body, date/time with its known precision, venue, lifecycle, poster/gallery/reports and an approved action link where one exists.
- **Person:** stable key, name, department, portrait and only approved public contacts/social links.
- **Membership:** person reference, group, academic year, optional role and display order. One person may appear in multiple groups or years.
- **Milestone:** original copy, verified date or year precision, display order and supported relationships.

- **Media:** use Shreyam's contract. Keep public delivery references separate from private masters/draft assets.
- **Globals:** home/About copy and shared site settings, including the official contact email and featured record references.
- **CMS user:** private login, active status and role. Never use this collection for the public team page.
- **Redirect:** exact old path, canonical internal destination and approved HTTP status.

Keep publication state separate from project/event lifecycle. A completed event can still be published. Keep the Drone Automation lifecycle as Ongoing and preserve the IEEE SSCS 2026 competition labels. Unknown facts remain unknown until the content owner resolves them.

5 B02: invite-only CMS accounts and permissions

Create `src/collections/Users.ts`, small access helpers in `src/access/`, a role matrix, and `tests/permissions.test.ts`.

Roles: Admin can publish/unpublish and manage accounts. Editor can create and revise drafts but cannot publish or grant roles. Anonymous visitors cannot create accounts, edit records, inspect CMS users or read drafts. Keep first-admin bootstrap private and limited to the chosen owner.

Steps:

1. Write role tests for anonymous, Editor and Admin before wiring the UI.
2. Protect collection read/create/update/delete operations and field-level user role changes.
3. Check the incoming changed data on create/update. Do not authorize an Editor just because the saved record previously had draft status.
4. Test direct API operations as well as the Admin UI; hiding a button does not protect the server.
5. Configure recovery only after the orchestrator assigns a club-owned email provider/account. Keep legacy Q&A credentials out of the new application.

Check: `pnpmtest--tests/permissions.test.ts`, then staging browser checks for login, logout and recovery. Attempt to self-promote an Editor and publish with a direct request; both must fail.

6 B03: Project collection

Create `src/collections/Projects.ts`, a migration, fixture and `tests/projects.test.ts`. Agree the public fields with Sourasish and Pritish; coordinate media references with Shreyam.

Keep lifecycle (for example, Ongoing or Completed) separate from award/program label and year. Preserve the original copy and established order. Require a unique slug for public detail routes. Allow incomplete authorized drafts, but validate required fields before publishing. Do not accept an arbitrary client-provided internal key as identity.

Check: test create/update, duplicate slug, invalid publish, saved draft while live version remains unchanged, unpublish, award label/year and changed slug. Send the agreed publication event to Sourasish with record type, ID, old/new slug and changed public relationships.

7 B04: Event collection

Create `src/collections/Events.ts`, migration and `tests/events.test.ts`. Keep date information precise: if the source only says a year or has no date, do not invent a full date. Define and document the timezone rule before deriving whether something is upcoming.

Separate editorial publication from the event lifecycle. A historical event can remain published. Add an action link only when approved by the content owner. Do not recreate expired registration/live actions or invent a countdown/Innovatia status.

Check: include completed-but-published, unknown date, date boundary, postponed/cancelled state and invalid external URL. Confirm the result does not change merely because a timezone differs.

8 B05: People and Memberships

Create `src/collections/People.ts`, `src/collections/Memberships.ts`, migrations and `tests/memberships.test.ts`. Keep one confirmed person with multiple linked memberships instead of copying a new person record for every team group.

Only merge identities after review. Similar names do not prove two entries are the same person. Preserve the verified department/role for every membership; retain Raunak Panja's Electrical Engineering and Ritendu's corrected department text. Alumni history belongs in verified memberships, not fabricated fields.

Check: one person shown in two groups retains both roles/orders; same-name different people stay distinct; deleting a referenced person is blocked or requires a safe relationship change. Coordinate with Shreyam on the source identity review.

9 B06a: Home, About and shared settings

Create `src/globals/Home.ts`, `src/globals/About.ts`, `src/globals/SiteSettings.ts`, migrations and focused tests. Agree exact fields with Pritish first. Store content and selected media/record references, not arbitrary custom HTML or page CSS.

Use one shared source for navigation/footer and the official email `iotajuofficial@gmail.com`. Do not select an unpublished or draft project/event as a public featured item. Notify Sourasish when changes must refresh cached public pages.

Check: change the contact email and a featured record in staging, verify every page consuming it, and verify anonymous reads return no draft or private media URL.

10 B06b: Milestones and Redirects

Create `src/collections/Milestones.ts`, `src/collections/Redirects.ts`, migrations and focused validation tests. Use Shreyam's source inventory. Sourasish owns redirect runtime.

Preserve only the source's actual date precision. Redirects must point to a canonical internal path; block obvious loops and external open redirects. Only authorized editors/admins may change these records.

Check: source-derived milestone fixtures, invalid destination, self-loop and a two-record loop. Coordinate deployed path tests with Sourasish.

11 B07: relationship safety and publication events

Create focused hooks under `src/hooks/`, reference guards and `tests/relationships.test.ts`. Work with Sourasish on when a publication notification is safe to send relative to a committed write.

Before publishing, validate selected media and linked records are approved and public. A nested relationship must not expose a private/draft CDN URL. On person, event, global or media change, list which public pages must refresh. Prevent hard deletion of referenced records or media unless references have been safely resolved.

Check: update a person shown in several groups; unpublish a featured item; revoke a media item; change a slug; confirm stale public links and media access are removed. Make hook failures visible to operators.

12 B08: editor guide and handover

After the pages and imports are available, create `docs/editor-guide.md` and `tests/e2e/editorial.spec.ts`. Rehearse with a club Editor: revise a project, event and member; preview a draft; have an Admin publish; restore a revision; unpublish; and hand the year's membership to another editor.

Explain how to prepare/upload approved media, add safe links, recover a failed publish and remove access when members leave. Do not promise that the app automatically transcodes large videos.

Check: an editor who did not write the guide completes the rehearsal without help. Confirm draft privacy through direct API requests, not only the visible page.

13 Rules for your changes

- Use Payload's generated types. Regenerate and commit them from the current config; never edit generated type output by hand.
- Put schema changes in committed migrations. Apply them to an isolated database first.
- Public local API calls must explicitly enforce access. Payload Local API defaults can bypass access rules when `overrideAccess` is not disabled; readers must not assume collection permissions automatically protect each server-side call.
- Do not create public user accounts, a second authentication service, a separate ORM or a new backend.
- Ask Sourasish before changing package/config/CI files. Do not overwrite Shreyam's Media adapter or import edits.

14 Official references

Use the official Payload installation guide, access control guide, Local API access-control guide, and database migration guide for version-specific details. The current guide requires Node.js 20.18.1 or newer. Check these again against the package version pinned by Sourasish.